

Perancangan dan Analisis Algoritma (A)

Tugas Pertemuan ke-8

Disusun Oleh:

Gilbran Mahdavia Raja

NRP: 5025241134

Institut Teknologi Sepuluh Nopember
Surabaya
2026

Contents

1	Studi Kasus 1: SPOJ TOHMOVE2 – Tower of Hanoi Movement (Hard)	2
1.1	Deskripsi Masalah	2
1.2	Mengapa Divide and Conquer?	2
1.3	Penurunan Relasi Rekurensi	2
1.4	Verifikasi Manual ($N = 3$)	3
1.5	Pseudocode	3
1.6	Kode Solusi (C++)	3
1.7	Analisis Kompleksitas	4
2	Studi Kasus 2: SPOJ VIENTIAN – Tower of Vientiane	4
2.1	Deskripsi Masalah	4
2.2	Mengapa Memoized Recursion?	4
2.3	Encoding Matriks ke Bitmask	5
2.4	Dua Strategi Rekurensi	5
2.4.1	Derivasi Strategi B	5
2.5	Relasi Rekurensi Lengkap	6
2.6	Verifikasi Manual	6
2.6.1	Kasus Uji 1: Matriks Hanoi Standar, $n = 5$	6
2.6.2	Kasus Uji 2: Matriks Terbatas, $n = 5$	6
2.6.3	Kasus Uji 3: Matriks Tanpa Jalur ke Tujuan, $n = 5$	7
2.7	Pseudocode	7
2.8	Kode Solusi (C++)	7
2.9	Analisis Kompleksitas	9
3	Perbandingan Dua Studi Kasus	9
4	Sintesis: Pola Berpikir	9
5	Kesimpulan	10
6	Lampiran	12

Studi Kasus 1: SPOJ TOHMOVE2 – Tower of Hanoi Movement (Hard)

1.1 Deskripsi Masalah

Diberikan Tower of Hanoi dengan N disk dan 3 rod (A sebagai sumber, B sebagai perantara, C sebagai tujuan). Solusi optimal membutuhkan $2^N - 1$ langkah. Diberikan bilangan a ($1 \leq a \leq 2^N - 1$), tentukan langkah ke- a : disk mana yang bergerak dan dari rod mana ke rod mana.

Batasan: $1 \leq T \leq 1000$, $1 \leq N \leq 50$, $1 \leq a \leq 2^N - 1$.

1.2 Mengapa Divide and Conquer?

Pendekatan naif — menghasilkan seluruh $2^N - 1$ langkah lalu mengambil langkah ke- a — tidak layak untuk $N = 50$ karena $2^{50} \approx 10^{15}$ langkah. Sebaliknya, struktur rekursif Tower of Hanoi memiliki sifat berikut:

Struktur Rekursif Tower of Hanoi

Untuk memindahkan n disk dari rod **src** ke rod **dst** menggunakan **aux** sebagai perantara:

1. **Fase 1** (langkah 1 s.d. $2^{n-1} - 1$): Pindahkan $n - 1$ disk dari **src** ke **aux**.
 2. **Fase 2** (langkah 2^{n-1}): Pindahkan disk ke- n dari **src** ke **dst**.
 3. **Fase 3** (langkah $2^{n-1} + 1$ s.d. $2^n - 1$): Pindahkan $n - 1$ disk dari **aux** ke **dst**.
- Setiap fase memiliki ukuran tepat $2^{n-1} - 1$, 1, dan $2^{n-1} - 1$.

Karena ukuran tiap fase diketahui secara pasti, kita dapat menentukan fase mana yang memuat langkah ke- a hanya dengan perbandingan, lalu merekursi ke submasalah yang sesuai tanpa mengeksekusi fase lain.

1.3 Penurunan Relasi Rekurensi

Didefinisikan:

$\text{solve}(n, a, \text{src}, \text{dst}, \text{aux})$ = langkah ke- a saat memindahkan n disk dari **src** ke **dst**

Dengan $\text{half} = 2^{n-1}$:

Relasi Rekurensi – TOHMOVE2

$$\text{solve}(n, a, \text{src}, \text{dst}, \text{aux}) = \begin{cases} \{n, \text{src} \rightarrow \text{dst}\} & \text{jika } a = \text{half} \\ \text{solve}(n-1, a, \text{src}, \text{aux}, \text{dst}) & \text{jika } a < \text{half} \\ \text{solve}(n-1, a - \text{half}, \text{aux}, \text{dst}, \text{src}) & \text{jika } a > \text{half} \end{cases}$$

Perhatikan pertukaran peran rod pada tiap cabang:

- Fase 1: **aux** dan **dst** bertukar peran (tujuan sementara adalah **aux**).
- Fase 3: **src** dan **aux** bertukar peran (sumber menjadi **aux**).

1.4 Verifikasi Manual ($N = 3$)

Solusi optimal $N = 3$ memiliki $2^3 - 1 = 7$ langkah. Rod diberi indeks: A=0, B=1, C=2.

Table 1: Seluruh langkah solusi optimal Tower of Hanoi $N = 3$

Langkah a	Disk	Dari	Ke	Keterangan
1	1	A	C	Fase 1 dari solve(3,...)
2	2	A	B	Fase 1 dari solve(3,...)
3	1	C	B	Fase 1 dari solve(3,...)
4	3	A	C	Fase 2: disk terbesar
5	1	B	A	Fase 3 dari solve(3,...)
6	2	B	C	Fase 3 dari solve(3,...)
7	1	A	C	Fase 3 dari solve(3,...)

Verifikasi langkah ke-5 ($N = 3$, $a = 5$):

solve(3, 5, A, C, B): half = 4, $a > 4 \Rightarrow$ solve(2, 1, B, C, A)

solve(2, 1, B, C, A): half = 2, $a < 2 \Rightarrow$ solve(1, 1, B, A, C)

solve(1, 1, B, A, C): half = 1, $a = 1 \Rightarrow \{1, B \rightarrow A\} \checkmark$

1.5 Pseudocode

Listing 1: Pseudocode TOHMOVE2

```
function solve(n, a, src, dst, aux):
    half = 2^(n-1)
    if a == half:
        return Move(disk=n, from=src, to=dst)
    elif a < half:
        return solve(n-1, a, src, aux, dst)    // Fase 1: dst dan
        aux bertukar
    else:
        return solve(n-1, a - half, aux, dst, src) // Fase 3: src
        dan aux bertukar
```

1.6 Kode Solusi (C++)

Listing 2: Solusi SPOJ TOHMOVE2 - C++

```
1 #include <cstdio>
2 const char rod[] = {'A', 'B', 'C'};
3 struct Move { int disk, src, dst; };
4
5 Move solve(int n, long long a, int src, int dst, int aux) {
6     long long half = 1LL << (n - 1);
7     if (a == half)
8         return {n, src, dst};
9     if (a < half)
10        return solve(n - 1, a, src, aux, dst);
```

```

11     else
12         return solve(n - 1, a - half, aux, dst, src);
13 }
14
15 int main() {
16     int T;
17     scanf("%d", &T);
18     while (T--) {
19         int N;
20         long long a;
21         scanf("%d %lld", &N, &a);
22         Move m = solve(N, a, 0, 2, 1);
23         printf("%d : %c => %c\n", m.disk, rod[m.src], rod[m.dst]);
24     }
25     return 0;
26 }

```

1.7 Analisis Kompleksitas

Table 2: Kompleksitas TOHMOVE2

Aspek	Nilai	Alasan
Waktu (per query)	$O(N)$	Rekursi turun tepat N level
Ruang (call stack)	$O(N)$	Kedalaman rekursi maksimal N
Waktu (total T query)	$O(T \cdot N)$	Tiap query independen

Tidak ada memoization yang diperlukan karena setiap query menelusuri satu jalur unik dari akar ke daun — tidak ada submasalah yang tumpang tindih dalam satu eksekusi query.

Studi Kasus 2: SPOJ VIENTIAN – Tower of Vientiane

2.1 Deskripsi Masalah

Tower of Vientiane adalah variasi Tower of Hanoi di mana pergerakan disk antar rod dibatasi oleh matriks izin 3×3 . Elemen ke- (i, j) bernilai 1 jika perpindahan dari rod i ke rod j diizinkan. Rod awal bernomor 1, rod tujuan bernomor 3, rod perantara bernomor 2. Tugas: temukan jumlah langkah minimum untuk memindahkan n disk dari rod 1 ke rod 3, atau nyatakan *Epic Fail*... jika mustahil.

Batasan: $1 \leq t \leq 10000$, $2 \leq n \leq 39$.

2.2 Mengapa Memoized Recursion?

Berbeda dengan Hanoi standar, banyaknya langkah tidak lagi selalu $2^n - 1$ karena bergantung pada struktur graf gerakan. Terdapat dua sifat kunci yang muncul:

- **Overlapping subproblems:** Pasangan $(mask, n, src, dst)$ yang sama dapat muncul pada berbagai jalur rekursi.

- **Optimal substructure:** Biaya minimum untuk n disk dibangun dari biaya minimum untuk $n - 1$ disk.

Karena itu, memoization digunakan untuk memastikan setiap state hanya dihitung satu kali.

2.3 Encoding Matriks ke Bitmask

Matriks 3×3 direpresentasikan sebagai integer 9-bit:

Encoding Bitmask

$$\text{mask} = \sum_{i=0}^2 \sum_{j=0}^2 \text{allow}[i][j] \cdot 2^{3i+j}$$

Cek apakah gerakan $i \rightarrow j$ diizinkan:

$$\text{allowed}(i, j) = (\text{mask} \gg (3i + j)) \& 1$$

Dengan encoding ini, seluruh variasi matriks gerakan terwakili oleh $2^9 = 512$ nilai mask yang berbeda.

2.4 Dua Strategi Rekurensi

Dua Strategi Pemindahan Disk Terbesar

Strategi A — Disk terbesar bergerak *langsung* dari **src** ke **dst** (1 langkah):
Syarat: $\text{allowed}(\text{src}, \text{dst}) = 1$. Pilih rod **aux** sebagai perantara $n - 1$ disk.

$$f(n, \text{src}, \text{dst}) = f(n - 1, \text{src}, \text{aux}) + 1 + f(n - 1, \text{aux}, \text{dst})$$

Strategi B — Disk terbesar harus *relay* melalui **via** (2 langkah: **src** $\rightarrow$ **via** $\rightarrow$ **dst**):

Syarat: $\text{allowed}(\text{src}, \text{via}) = 1$ dan $\text{allowed}(\text{via}, \text{dst}) = 1$.

$$f(n, \text{src}, \text{dst}) = 2 \cdot f(n - 1, \text{src}, \text{dst}) + f(n - 1, \text{dst}, \text{src}) + 2$$

2.4.1 Derivasi Strategi B

Karena disk terbesar tidak bisa langsung ke tujuan, diperlukan 5 fase:

Table 3: Lima fase pada Strategi B

Fase	Operasi	Biaya	Tujuan
1	Pindah $n - 1$ disk: src $\rightarrow$ dst	$f(n - 1, \text{src}, \text{dst})$	Kosongkan src
2	Pindah disk ke- n : src $\rightarrow$ via	1	Geser besar ke via
3	Pindah $n - 1$ disk: dst $\rightarrow$ src	$f(n - 1, \text{dst}, \text{src})$	Kosongkan dst
4	Pindah disk ke- n : via $\rightarrow$ dst	1	Letakkan besar di dst
5	Pindah $n - 1$ disk: src $\rightarrow$ dst	$f(n - 1, \text{src}, \text{dst})$	Tumpuk kembali
Total		$2 \cdot f(n - 1, \text{src}, \text{dst}) + f(n - 1, \text{dst}, \text{src}) + 2$	

2.5 Relasi Rekurensi Lengkap

Relasi Rekurensi – VIENTIAN

$$f(\text{mask}, n, \text{src}, \text{dst}) = \min \left(\underbrace{\min_{\text{aux}} [f_{n-1}^{\text{src}, \text{aux}} + 1 + f_{n-1}^{\text{aux}, \text{dst}}]}_{\text{Strategi A (jika allowed)}}, \underbrace{\min_{\text{via}} [2f_{n-1}^{\text{src}, \text{dst}} + f_{n-1}^{\text{dst}, \text{src}} + 2]}_{\text{Strategi B (jika via tersedia)}} \right)$$

dengan notasi $f_{n-1}^{p,q} = f(\text{mask}, n - 1, p, q)$.

Kasus dasar: $f(\text{mask}, 0, *, *) = 0$ dan $f(\text{mask}, n, x, x) = 0$.

Jika tidak ada strategi yang valid: $f = \infty$ (Epic Fail...).

2.6 Verifikasi Manual

2.6.1 Kasus Uji 1: Matriks Hanoi Standar, $n = 5$

Matriks: semua gerakan diizinkan kecuali diagonal. Karena jalur langsung selalu ada, Strategi A selalu berlaku dan menghasilkan $2^5 - 1 = 31$ langkah. ✓

2.6.2 Kasus Uji 2: Matriks Terbatas, $n = 5$

$$M = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad (\text{hanya: } 1 \rightarrow 2, 2 \rightarrow 1, 2 \rightarrow 3, 3 \rightarrow 2)$$

Tidak ada jalur langsung $1 \rightarrow 3$, sehingga hanya Strategi B yang berlaku dengan **via** = rod 2. Rekurensi menghasilkan pola $f(n) = 3^n - 1$:

Table 4: Tabulasi $f(n, \text{rod}_1, \text{rod}_3)$ untuk matriks terbatas

n	$f(n - 1, 1, 3)$	$f(n - 1, 3, 1)$	$f(n) = 2f_{n-1}^{1,3} + f_{n-1}^{3,1} + 2$
1	—	—	2
2	2	2	8
3	8	8	26
4	26	26	80
5	80	80	242

Output yang diharapkan: 242. ✓

2.6.3 Kasus Uji 3: Matriks Tanpa Jalur ke Tujuan, $n = 5$

$$M = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Tidak ada jalur dari rod 3 ke rod 1, sehingga Strategi B tidak bisa membalik $n - 1$ disk. Hasil: Epic Fail.... ✓

2.7 Pseudocode

Listing 3: Pseudocode VIENTIAN – get_cost

```
function get_cost(mask, n, src, dst):
    if n == 0 or src == dst: return 0
    if visited[mask][n][src][dst]: return memo[mask][n][src][dst]

    mark visited[mask][n][src][dst] = true
    best = INF

    // Strategi A
    if allowed(src, dst):
        for each aux in {0,1,2} \ {src, dst}:
            c1 = get_cost(mask, n-1, src, aux)
            c2 = get_cost(mask, n-1, aux, dst)
            if c1 < INF and c2 < INF:
                best = min(best, c1 + 1 + c2)

    // Strategi B
    for each via in {0,1,2} \ {src, dst}:
        if allowed(src, via) and allowed(via, dst):
            c1 = get_cost(mask, n-1, src, dst)
            c3 = get_cost(mask, n-1, dst, src)
            if c1 < INF and c3 < INF:
                best = min(best, 2*c1 + c3 + 2)

    memo[mask][n][src][dst] = best
    return best
```

2.8 Kode Solusi (C++)

Listing 4: Solusi SPOJ VIENTIAN – C++

```
1 #include <cstdio>
2 const long long INF = 4611686018427387904LL;
3 long long memo[512][40][3][3];
4 bool      vis [512][40][3][3];
5
6 long long solve(int mask, int n, int src, int dst) {
7     if (n == 0 || src == dst) return 0;
```



```

8   if (vis[mask][n][src][dst]) return memo[mask][n][src][dst];
9   vis[mask][n][src][dst] = true;
10  long long best = INF;
11
12  if ((mask >> (src * 3 + dst)) & 1) {
13      for (int aux = 0; aux < 3; aux++) {
14          if (aux == src || aux == dst) continue;
15          long long c1 = solve(mask, n - 1, src, aux);
16          if (c1 >= INF) continue;
17          long long c2 = solve(mask, n - 1, aux, dst);
18          if (c2 >= INF) continue;
19          long long t = c1 + 1 + c2;
20          if (t < best) best = t;
21      }
22  }
23  for (int via = 0; via < 3; via++) {
24      if (via == src || via == dst) continue;
25      if (!((mask >> (src * 3 + via)) & 1)) continue;
26      if (!((mask >> (via * 3 + dst)) & 1)) continue;
27      long long c1 = solve(mask, n - 1, src, dst);
28      if (c1 >= INF) continue;
29      long long c3 = solve(mask, n - 1, dst, src);
30      if (c3 >= INF) continue;
31      long long t = 2 * c1 + c3 + 2;
32      if (t < best) best = t;
33  }
34  memo[mask][n][src][dst] = best;
35  return best;
36 }
37
38 int main() {
39     int t;
40     scanf("%d", &t);
41     while (t--) {
42         int mask = 0;
43         for (int i = 0; i < 3; i++) {
44             char row[8];
45             scanf("%s", row);
46             for (int j = 0; j < 3; j++)
47                 if (row[j] == '1')
48                     mask |= (1 << (i * 3 + j));
49         }
50         int n;
51         scanf("%d", &n);
52         long long result = solve(mask, n, 0, 2);
53         if (result >= INF) printf("Epic Fail...\n");
54         else printf("%lld\n", result);
55     }
56     return 0;
57 }

```

2.9 Analisis Kompleksitas

Table 5: Kompleksitas VIENTIAN

Aspek	Nilai	Alasan
Jumlah state unik	$512 \times 40 \times 3 \times 3 = 184,320$	Dimensi array memo
Waktu per state	$O(1)$	Loop maksimal 3 iterasi
Waktu total (worst case)	$O(184,320)$	Semua state dihitung sekali
Ruang	$O(184,320)$	Array memo dan vis global
Waktu per query	$O(N)$	Hanya lookup + navigasi

Karena `memo` dan `vis` adalah variabel global, keduanya di-*zero-initialize* secara otomatis oleh runtime C++ sebelum `main` dipanggil. `memset` tidak diperlukan. Cache juga dipakai bersama lintas seluruh test case, sehingga query berulang dengan `mask` dan `n` yang sama tidak dihitung ulang.

Perbandingan Dua Studi Kasus

Table 6: Perbandingan TOHMOVE2 dan VIENTIAN

Aspek	TOHMOVE2	VIENTIAN
Masalah	Query langkah ke- a	Jumlah langkah minimum
Teknik	D&C murni	Memoized recursion
Memoization	Tidak	Ya (global 4D array)
Jumlah fase	3	3 (Strategi A) atau 5 (Strategi B)
Jumlah langkah	Selalu $2^n - 1$	Bergantung pada matriks
Kasus tak mungkin	Tidak ada	Epic Fail... jika $= \infty$
Rekursi kunci	$\text{half} = 2^{n-1}$	$2f_{n-1} + f'_{n-1} + 2$
Waktu per query	$O(N)$	$O(N)$ setelah cache terisi
Tipe data	long long	long long
Nilai maksimum	$2^{50} - 1 \approx 10^{15}$	Bergantung mask, bisa besar

Sintesis: Pola Berpikir

Hierarki Teknik yang Dipelajari

1. **Kenali struktur rekursif** — apakah masalah terbagi menjadi submasalah dengan ukuran lebih kecil dan struktur identik?
2. **Tentukan parameter posisi** — untuk query posisi ke- a , hitung batas tiap fase agar dapat menentukan cabang rekursi tanpa mengeksekusi fase lain.
3. **Cek overlapping subproblems** — jika parameter state terbatas (seperti `mask`, n , `src`, `dst`), gunakan memoization.

4. **Identifikasi kasus tak mungkin** — gunakan sentinel ∞ dan propagasi dengan pemeriksaan `if (c >= INF) continue`.
5. **Pilih encoding efisien** — matriks boolean 3×3 lebih efisien disimpan sebagai bitmask 9-bit daripada array 2D.
6. **Verifikasi manual** — selalu uji rekurensi pada kasus kecil ($n = 1, 2, 3$) sebelum submit.

Kesimpulan

1. **D&C berbasis posisi tidak perlu menghasilkan seluruh solusi.** Dengan mengetahui ukuran tiap fase secara analitik, query posisi ke- a diselesaikan dalam $O(N)$ meski 2^N langkah tidak feasibel dihasilkan seluruhnya.
2. **Pembatasan graf gerakan memunculkan strategi relay.** Ketika jalur langsung tidak ada, disk terbesar harus bergerak dua kali melalui rod perantara, menghasilkan pola 5 fase dengan biaya $2f_{n-1}^{A,B} + f_{n-1}^{B,A} + 2$ yang secara aljabar setara dengan $3^n - 1$ untuk kasus linear.
3. **Memoization dengan bitmask mengatasi ledakan ruang state.** Encoding matriks 3×3 ke integer 9-bit membatasi dimensi pertama array hanya 512 entri, sehingga total state tetap di bawah 200 ribu meski t mencapai 10.000.
4. **Pemilihan sentinel yang tepat mencegah overflow.** Menggunakan $\infty = \lfloor \text{LLONG_MAX}/2 \rfloor$ memastikan operasi $2 \cdot \infty$ tidak overflow sehingga pemeriksaan `c >= INF` tetap benar.

References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [2] Rosen, K. H. (2012). *Discrete Mathematics and Its Applications* (7th ed.). McGraw-Hill. Bab 8: Linear Recurrence Relations.
- [3] SPOJ. (n.d.). *TOHMOVE2 – Tower of Hanoi Movement (Hard)*. <https://www.spoj.com/problems/TOHMOVE2/>
- [4] SPOJ. (n.d.). *VIENTIAN – Tower of Vientiane*. <https://www.spoj.com/problems/VIENTIAN/>

Lampiran

GilbranNRP134: submissions Tower of Vientiane

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35697908	2026-04-22 07:02:50	Tower of Vientiane	accepted edit ideone.it	0.01	5.2M	CPP14

Figure 1: Accepted submission – SPOJ VIENTIAN

GilbranNRP134: submissions Tower of Hanoi Movement - Hard

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35697849	2026-04-22 06:42:29	Tower of Hanoi Movement - Hard	accepted edit ideone.it	0.04	5.2M	CPP14

Figure 2: Accepted submission – SPOJ TOHMOVE2